

LAPORAN TUGAS
WEB SERVER
(TIF 330805)
MINGGU KE 8



ACARA 14
DIBUAT OLEH:
NAMA: CINTYA AZWA SAFRINA R.H
NIM: E41250631
GOL: A

**PROGRAM STUDI DI LUAR KAMPUS UTAMA (PSDKU) POLITEKNIK NEGERI
JEMBER**
**TEKNIK INFORMATIKA KAB. NGANJUK JL. JENDRAL GATOT SUBROTO NO
106, KAUMAN, KECAMATAN NGANJUK, KABUPATEN NGANJUK**
**JURUSAN TEKNOLOGI INFORMASI POLITEKNIK NEGERI JEMBER TAHUN
2026**

PENDAHULUAN

Aplikasi web tidak hanya harus berjalan ketika semuanya normal, tetapi juga harus tetap aman ketika terjadi kesalahan. Salah satu kesalahan yang paling sering terjadi adalah gangguan pada database, misalnya server MySQL mati atau koneksi ditolak. Tanpa penanganan yang baik, PHP menampilkan pesan error mentah kepada pengguna, berisi lokasi folder di server, urutan pemanggilan kode (stack trace), bahkan nama pengguna database. Informasi ini berbahaya karena dapat dimanfaatkan pihak yang tidak bertanggung jawab.

Solusinya adalah Exception Handling, yaitu menangkap error dengan blok try/catch lalu menggantinya dengan pesan yang ramah dan aman bagi pengguna, misalnya "Terjadi kesalahan pada sistem. Silakan coba lagi nanti." Detail teknis error tidak dibuang begitu saja, melainkan dicatat ke dalam file log (Logging). Sebuah log yang baik menjawab tiga pertanyaan: kapan error terjadi, error apa yang terjadi, dan di bagian aplikasi mana. Dengan demikian, pengembang tetap dapat menelusuri penyebab masalah tanpa membuka informasi sensitif kepada pengguna.

Selain itu, praktikum ini juga membahas beberapa praktik pengamanan yang sudah diterapkan pada Acara 13, yaitu Prepared Statement untuk mencegah SQL Injection, validasi input dan pengecekan NIM duplikat di Service Layer, Flash Message untuk umpan balik pengguna, serta pola PRG (Post-Redirect-Get) agar data tidak tersimpan ganda ketika halaman di-refresh setelah pengiriman form.

Praktikum ini menambahkan exception handling dan logging pada aplikasi Sistem Informasi Akademik, kemudian menguji perilakunya melalui simulasi error database.

IMPLEMENTASI

1. Menyimpan Acara 13 dan menyiapkan branch Acara 14

```
git add .
git commit -m "Menambahkan MahasiswaService dan flash message"
git push origin acara-13
```

```
git checkout -b acara-14
Copy-Item -Path Acara_13 -Destination Acara_14 -Recurse
```

Pekerjaan Acara 13 di-commit dan di-push terlebih dahulu agar riwayat Git tiap acara terpisah. Praktikum dikerjakan pada branch acara-14, dan folder Acara_13 disalin menjadi Acara_14 sebagai dasar pengerjaan.

2. Memeriksa kondisi project dari Acara 13

Sebagian besar materi Acara 14 sudah tersedia pada project Acara 13. Status masing-masing materi ditunjukkan pada tabel berikut.

Materi	Status di project
Prepared Statement	Sudah, semua query Repository memakainya
Validasi input dan NIM duplikat	Sudah, ada di MahasiswaService
Flash Message	Sudah
PRG (POST, redirect, GET)	Sudah, perlu dibuktikan lewat uji refresh
Exception Handling	Belum lengkap, error database belum ditangkap di Controller
Logging ke storage/logs/app.log	Belum ada
Simulasi error database dan pengecekan log	Belum dilakukan

Karena itu pekerjaan difokuskan pada exception handling dan logging, kemudian pengujian skenario.

3. Membuat Logger

Folder penyimpanan log dibuat terlebih dahulu dengan perintah `New-Item -ItemType Directory -Path Acara_14\storage\logs`. Setelah itu dibuat file `app\Helpers\Logger.php`.

```
<?php
class Logger
{
    private static function path(): string
    {
        return __DIR__ . '/../../storage/logs/app.log';
    }

    public static function error(string $context, Throwable $e): void
    {
        $dir = dirname(self::path());
```

```

        if (!is_dir($dir)) {
            mkdir($dir, 0777, true);
        }

        $line = date('Y-m-d H:i:s')
            . ' - [' . $context . ']' . ' '
            . get_class($e) . ': ' . $e->getMessage()
            . PHP_EOL;

        error_log($line, 3, self::path());
    }
}

```

Method `Logger::error()` menulis satu baris ke `app.log` berisi waktu kejadian, bagian aplikasi (`$context`), jenis exception, dan pesan error. Hanya pesan exception yang dicatat sehingga password database tidak ikut tertulis, dan folder logs dibuat otomatis bila belum ada. File `.gitignore` berisi `/storage/logs/*`, sehingga `app.log` tidak ikut ke GitHub.

4. Memasang exception handling dan logging di MahasiswaService

Ada dua perubahan dibanding versi Acara 13. Pertama, seluruh proses (termasuk validasi yang memanggil database) dibungkus try/catch, karena sebelumnya `existsByNim()` dan `exists()` berada di luar try sehingga error database di situ tidak tertangkap. Kedua, setiap exception dicatat melalui Logger, sedangkan pengguna hanya menerima pesan aman. Ditambahkan juga method `delete()` agar proses hapus melalui Service.

```

public function create(array $input): array
{
    try {
        $errors = $this->validate($input);
        if (!empty($errors)) {
            return ['success' => false, 'errors' => $errors];
        }

        $this->repo->create($this->buatEntity($input));
        return ['success' => true, 'errors' => []];
    } catch (PDOException $e) {
        Logger::error('MahasiswaService::create', $e);
        if ($e->getCode() === '23000') {
            return ['success' => false, 'errors' => ['NIM sudah terdaftar']];
        }
        return ['success' => false, 'errors' => ['Data gagal disimpan']];
    } catch (Throwable $e) {
        Logger::error('MahasiswaService::create', $e);
        return ['success' => false, 'errors' => ['Data gagal disimpan']];
    }
}

public function delete(int $id): array
{
    try {
        $this->repo->delete($id);
        return ['success' => true, 'errors' => []];
    } catch (Throwable $e) {

```

```

        Logger::error('MahasiswaService::delete', $e);
        return ['success' => false, 'errors' => ['Data gagal dihapus']];
    }
}

```

Blok catch (PDOException \$e) menangani error database. Kode 23000 adalah pelanggaran aturan UNIQUE (NIM kembar yang lolos pengecekan, misalnya dua pengguna menyimpan bersamaan). Blok catch (Throwable \$e) menjadi pengaman terakhir untuk error apa pun. Method update() dibungkus dengan pola yang sama.

5. Mengubah MahasiswaController

Controller diubah pada tiga hal. Pertama, destroy() memakai \$this->service->delete(\$id) dan hasilnya dikirim sebagai Flash Message. Kedua, method yang membaca data (index, create, edit) dibungkus try/catch dan memanggil method baru tampilkanError(), yang mencatat log lalu menampilkan pesan aman. Ketiga, store(), update(), dan destroy() tetap diakhiri redirect sehingga pola PRG terjaga.

```

private function tampilkanError(string $context, Throwable $e): void
{
    Logger::error($context, $e);
    http_response_code(500);
    echo "Terjadi kesalahan pada sistem. Silakan coba lagi nanti.";
}

public function destroy(int $id): void
{
    $result = $this->service->delete($id);

    if ($result['success']) {
        Flash::set('success', 'Data mahasiswa berhasil dihapus');
    } else {
        Flash::set('danger', implode(', ', $result['errors']));
    }

    $this->redirect("{ $this->base() } /mahasiswa");
}

```

Method store() dan update() tidak memerlukan try/catch karena Service sudah menangkap seluruh error dan mengembalikannya dalam bentuk array.

6. Menutup celah pada public/index.php

Pada simulasi pertama (MySQL dimatikan), ditemukan celah: error PDOException muncul sebagai pesan mentah. Penyebabnya, koneksi database dibuat di routes/web.php melalui Database::getInstance() ketika objek Repository dirangkai, yaitu sebelum Controller dijalankan. Akibatnya try/catch di Controller dan Service tidak pernah menangkapnya dan Logger tidak terpanggil. Celah ditutup dengan membungkus pemuatan routes/web.php pada front controller.

```
<?php
```

```
if (session_status() === PHP_SESSION_NONE) {  
    session_start();  
}  
  
require_once __DIR__ . '/../app/Helpers/Logger.php';  
  
try {  
    require_once __DIR__ . '/../routes/web.php';  
} catch (Throwable $e) {  
    Logger::error('public/index.php', $e);  
    http_response_code(500);  
    echo 'Terjadi kesalahan pada sistem. Silakan coba lagi nanti.';  
}
```

Dengan perubahan ini, error apa pun yang lolos dari lapisan di bawahnya tetap tertangkap, dicatat dengan konteks public/index.php, dan diganti pesan aman.

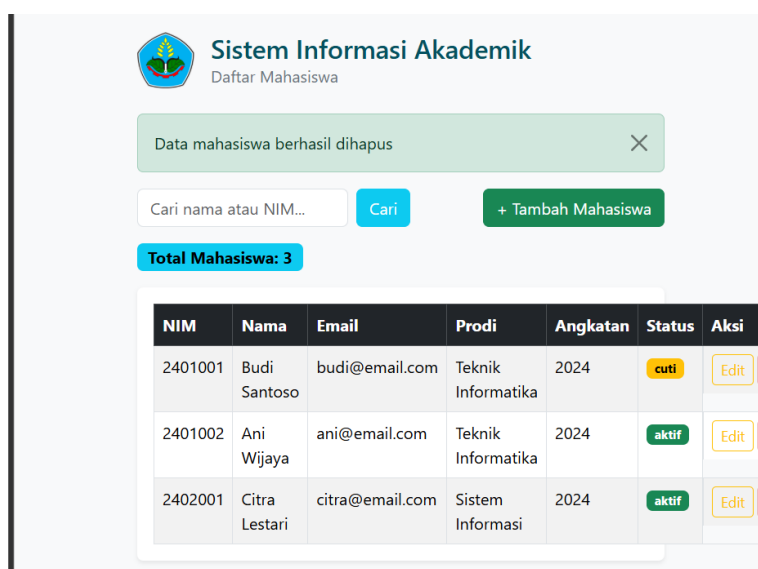
HASIL PENGUJIAN

Seluruh proses diuji melalui browser pada halaman Daftar Mahasiswa (localhost/tugas_acara_cya/Acara_14/public/mahasiswa). Ringkasan pengujian ditampilkan pada tabel berikut.

No	Skenario	Hasil yang Diharapkan	Hasil Pengujian
1	Tambah data valid	Redirect ke daftar, flash hijau "berhasil ditambahkan"	[belum diuji]
2	Tambah dengan NIM kosong	Flash merah "NIM wajib diisi"	[belum diuji]
3	Tambah dengan NIM berisi huruf	Flash merah "NIM harus berupa angka"	[belum diuji]
4	Tambah dengan NIM duplikat	Flash merah "NIM sudah terdaftar"	[belum diuji]
5	Tambah dengan email salah	Flash merah "Format email tidak valid"	[belum diuji]
6	Ubah data valid	Flash hijau "berhasil diubah"	[belum diuji]
7	Ubah NIM menjadi NIM mahasiswa lain	Flash merah "NIM sudah terdaftar"	[belum diuji]
8	Hapus data	Flash hijau "berhasil dihapus"	Sesuai
9	Uji PRG: tekan F5 setelah tambah data valid	Tidak ada dialog resubmission, data tidak dobel	[belum diuji]
10	Error database (MySQL dimatikan)	Pesan aman di layar dan baris di app.log	Sesuai

a. Hapus data (Skenario 8)

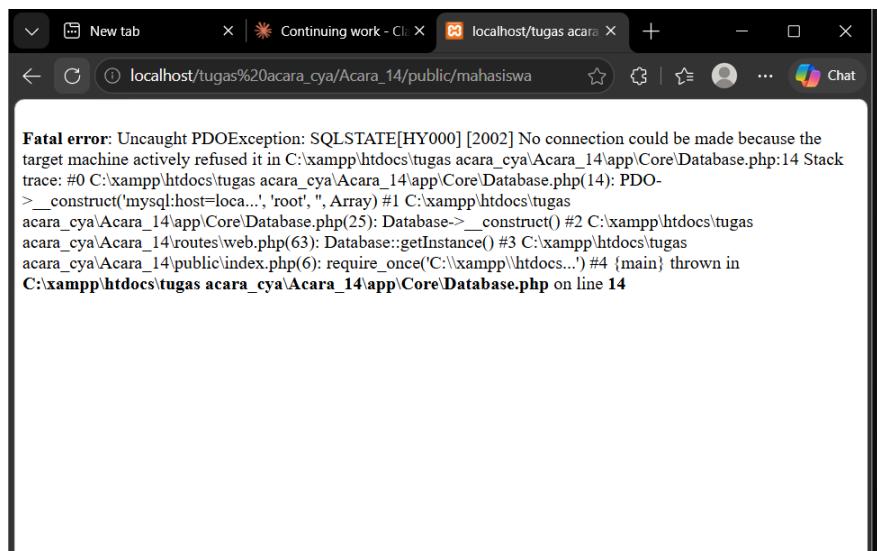
Setelah Controller diubah, proses hapus berjalan melalui service->delete(). Pengguna dikembalikan ke halaman daftar dengan Flash Message hijau "Data mahasiswa berhasil dihapus", daftar tampil normal dengan 3 data, dan tidak ada error PHP mentah di layar.



Gambar 1. Flash Message "Data mahasiswa berhasil dihapus"

b. Error database sebelum perbaikan (Skenario 10)

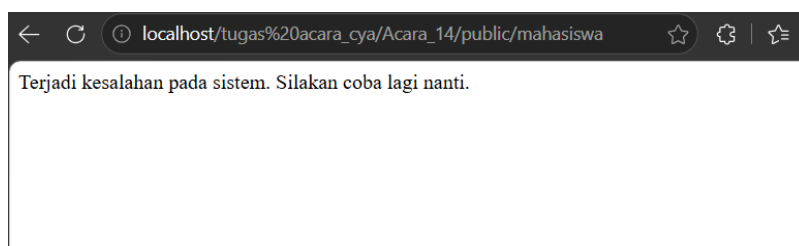
MySQL dimatikan melalui XAMPP Control Panel, lalu halaman /mahasiswa dimuat ulang. Layar menampilkan Fatal error: Uncaught PDOException yang membocorkan lokasi folder di server, urutan pemanggilan kode, dan username database (root). Error terjadi di routes/web.php baris 63, sebelum Controller berjalan, sehingga Logger tidak terpanggil dan app.log tidak terbentuk.



Gambar 2. Error mentah sebelum perbaikan index.php

c. Error database setelah perbaikan (Skenario 10)

Setelah index.php dibungkus try/catch, pengujian diulang dengan MySQL tetap mati. Layar hanya menampilkan pesan aman, tanpa lokasi folder, stack trace, maupun username database.



Gambar 3. Pesan aman setelah perbaikan

d. Isi app.log (Skenario 10)

Pengecekan dengan perintah Get-Content Acara_14\storage\logs\app.log menunjukkan satu baris log yang menjawab tiga pertanyaan: kapan (2026-10-08 03:11:37), error apa (PDOException, koneksi ditolak), dan di bagian mana (public/index.php). Tidak ada password atau lokasi folder yang tercatat.

```
PS C:\xampp\htdocs\tugas_acara_cya> Get-Content Acara_14\storage\logs\app.log
2026-10-08 03:11:37 - [public/index.php] PDOException: SQLSTATE[HY000] [2002] No connection could be made because the target machine actively refused it
PS C:\xampp\htdocs\tugas_acara_cya>
```

Gambar 4. Isi storage/logs/app.log

ANALISIS

Sebelum praktikum ini, error database hanya tertangkap sebagian. Pemanggilan `existsByNim()` dan `exists()` di Service berada di luar try, dan Controller tidak memiliki penanganan error sama sekali. Setelah perubahan, seluruh proses di Service dibungkus try/catch, method baca di Controller memakai `tampilkanError()`, dan `index.php` menjadi lapisan terakhir yang menangkap error apa pun.

Pengujian dengan mematikan MySQL menunjukkan bahwa penanganan error di Controller dan Service saja belum cukup. Koneksi database ternyata dibuat di `routes/web.php`, yaitu pada tahap perangkaian objek (Composition Root) sebelum Controller dipanggil. Error pada tahap ini lolos dan menampilkan stack trace lengkap berisi lokasi folder dan username database. Setelah pemuatan `routes/web.php` dibungkus try/catch di `index.php`, tampilan berubah menjadi pesan aman. Hal ini membuktikan bahwa exception handling harus mencakup seluruh alur permintaan, termasuk pembuatan koneksi.

Logging berfungsi sebagai pengganti pesan error yang disembunyikan dari pengguna. Baris log berisi waktu, jenis exception, dan konteks, sehingga pengembang tetap dapat menelusuri penyebab masalah. Karena yang dicatat hanya pesan exception, password database tidak ikut tertulis. File `app.log` juga dikecualikan dari Git melalui `.gitignore` agar log yang bisa memuat informasi sensitif tidak terunggah ke GitHub.

Pemisahan pesan juga penting: pengguna menerima pesan umum yang aman ("Data gagal disimpan" atau "Terjadi kesalahan pada sistem"), sedangkan detail teknis hanya masuk ke log. Pesan khusus tetap diberikan untuk kasus yang memang perlu diketahui pengguna, yaitu NIM duplikat (kode 23000).

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Exception handling dengan try/catch mencegah pesan error mentah (lokasi folder, stack trace, username database) tampil kepada pengguna, dan menggantinya dengan pesan yang aman.
2. Logging ke storage/logs/app.log mencatat kapan error terjadi, error apa, dan di bagian aplikasi mana, tanpa mencatat password database.
3. Penanganan error harus mencakup seluruh alur permintaan. Koneksi database yang dibuat di routes/web.php sebelum Controller berjalan hanya dapat ditangkap di index.php.
4. Pesan untuk pengguna dan detail teknis dipisahkan: pengguna menerima pesan umum, sedangkan detail teknis disimpan di log.
5. File app.log dikecualikan dari repository melalui .gitignore karena log dapat memuat informasi sensitif.